Social Media Content Planner – Project Report

Module: CSC1024 / PRG3024 / FEL1184 Programming Principles
Institution: Sunway University
Academic Session: May 2026

Group Members:

- Amos Lo Jun Hao (ID: 25018961) - Group Leader
- Law Yuan Rui (ID: 26035766)
- Wong Xin Er (ID: 23088320)
- Arooba Shahid (ID: 26011023)

---

1. Executive Summary & Overview

Social media management has become an essential activity for content creators, student influencers, and digital marketers. Keeping track of post ideas, scheduling content across different social media platforms, and measuring post engagement can easily become chaotic without a central tool.

The Social Media Content Planner is a user-friendly, text-based Python application built to solve this problem. It allows users to plan, schedule, update, track, and analyze their social media posts from a single menu interface without needing any complicated external libraries or software. All data is saved automatically into standard text files so that no progress is lost when the program closes.

The system was designed strictly following core programming principles, including clean modular functions, structured loops, conditional logic, file handling, and thorough input validation.

---

2. System Design and Architecture

2.1 High-Level Architecture

The system is divided into three simple connected layers:

1. **User Interface Layer:** The main menu loop displays options on the screen, accepts user choices, and prints clear messages.
2. **Business Logic & Validation Layer:** This layer processes post statuses, sorts posts chronologically for the content calendar, computes engagement analytics, and checks user inputs

to prevent errors.
3. **Data Storage Layer:** This layer handles reading and writing data to persistent text files on disk.

2.2 Data Files and Storage Format

The system uses three main text files to store data persistently. Each piece of information in a line is separated by a pipe symbol (`|`).

- **platforms.txt:** Stores information about supported social media platforms. Each line contains the platform ID, platform name, and total follower count. For example, a line might store "P1|Instagram|12500".
- **posts.txt:** Stores all post ideas created by the user. Each line contains the post ID, target platform, content caption, scheduled date, and current status. For example, a line might store "POST001|Instagram|New product launch!|2026-08-01|Scheduled".
- **engagement.txt:** Stores performance numbers for published posts. Each line contains the post ID followed by four numbers: likes, comments, shares, and views. For example, a line might store "POST002|1200|85|45|15000".

2.3 Post Life Cycle and State Management

Every post follows a clear life cycle consisting of three stages:

- **Draft:** New post ideas start as drafts by default.
- **Scheduled:** Posts that have been reviewed and given a publication date are moved to scheduled status.
- **Posted:** Once content goes live on social media, its status is updated to posted.

The system enforces a rule that engagement metrics can only be logged for posts that have reached the "Posted" status, ensuring data accuracy.

2.4 Modular Function Structure

The program source code in `planner.py` is broken down into clear, reusable functions:

- **File Handling Functions:** Functions such as `load_platforms()` , `load_posts()` , `save_posts()` , `load_engagement()` , and `save_engagement()` manage reading from and saving to the text files.
- **Input Validation Functions:** Functions like `get_valid_date()` and `get_non_negative_int()` handle user inputs safely to make sure dates and numbers are entered correctly.
- **Feature Functions:** Functions such as `add_new_post_idea()` , `update_post_status()` , `record_engagement_metrics()` , `display_content_calendar()` , and

`delete_post()` implement the core user actions.

- **Reporting Functions:** Functions like `compile_performance_report_data()`, `generate_performance_report()`, and `export_report_to_file()` summarize the engagement data and write summary reports to text files.

2.5 Sorting and Analytics Logic

- **Calendar Sorting:** To display the content calendar in order, the program uses a standard Selection Sort algorithm. It compares the date strings formatted as YYYY-MM-DD. Because this format lists the year first, month second, and day third, standard text comparison automatically sorts the dates from earliest to latest.
- **Performance Analytics:** Total engagement for a post is calculated by adding up its likes, comments, shares, and views. The reporting functions scan through all posts to figure out how many posts exist for each platform, which single post received the highest total engagement, and which platform received the most overall interactions.

---

3. Implementation Challenges and Solutions

3.1 Preventing File Delimiter Corruption

- **Challenge:** The application uses pipe symbols (`|`) to separate data fields in text files. If a user typed a pipe symbol inside a caption (for example, "Check out our site | Link in bio"), the file reader would get confused by the extra split parts and cause the program to crash or read data incorrectly.
- **Solution:** We added an input check when creating a post caption. If the user types a pipe character, the system displays an error message explaining that pipe symbols are reserved for system file formatting and prompts them to type the caption again.

3.2 Validating Dates Without External Packages

- **Challenge:** We needed to ensure users entered valid dates in the correct YYYY-MM-DD format without relying on complex external software libraries.
- **Solution:** We created a helper function named `get_valid_date()`. It splits the entered string by dashes and verifies that there are exactly three parts, that all parts contain numbers of the proper length (4 digits for year, 2 digits for month, 2 digits for day), and that the year, month, and day numbers fall within realistic ranges. It also lets users type 'cancel' to return to the main menu safely.

3.3 Cleaning Up Related Data When a Post is Deleted

- **Challenge:** Deleting a post from `posts.txt` left leftover engagement numbers in `engagement.txt` for that same post ID. Over time, these orphaned entries would waste space and could mess up future calculations if a post ID was reused.
- **Solution:** We updated the `delete_post()` function so that whenever a post is deleted, the program automatically searches through `engagement.txt` and removes any matching engagement record as well. This keeps all data files consistent automatically.

3.4 Preventing Invalid Metric Logged Entries

- **Challenge:** Users might try to enter likes or views for post ideas that were still in "Draft" or "Scheduled" status, which does not make sense logically.
- **Solution:** Before allowing engagement metrics to be entered, the system checks the status of the target post. If the post is not set to "Posted", the program displays a clear message explaining that metrics can only be recorded for published posts and stops the process.

---

4. Analysis of System Strengths and Limitations

4.1 System Strengths

- **No Extra Dependencies:** Built entirely with standard Python, meaning anyone can download and run the code immediately on Windows, Mac, or Linux without installing extra tools.
- **Human-Readable Storage:** Because data is stored in plain text files, users can easily view or back up their data using any basic text editor.
- **Crash-Proof Design:** All user prompts use validation loops that catch incorrect inputs, preventing accidental program crashes.
- **Automatic Data Cleanup:** Cascading deletion ensures that deleting a post also cleans up its corresponding performance data.
- **Tested Code:** Automated unit tests in `test_planner.py` verify that data loading and performance calculations work correctly.

4.2 System Limitations

- **File Rewrite Overhead:** Every time a post is added, updated, or deleted, the system rewrites the entire text file. While fast for small lists, this would slow down if managing tens of thousands of posts.
- **Single-User Access:** The text files do not support multiple users editing data at the exact same time, which could cause files to overwrite each other.
- **Text-Only Interface:** The program runs inside a command-line terminal window and does not feature visual elements like graphic buttons, interactive calendar grids, or visual charts.

- **Manual Data Entry:** Engagement numbers must be typed in manually by the user rather than being pulled automatically from social media accounts.

---

5. Suggestions for Future Improvements and Enhancements

5.1 Upgrading to a Relational Database

Replacing text files with a lightweight database like SQLite would improve performance and data management. Databases automatically handle relationships between tables, process large amounts of data faster, and prevent data corruption when multiple actions happen at once.

5.2 Creating a Graphical User Interface or Web Application

Building a graphical user interface using PyQt or converting the system into a web application using Flask and HTML/CSS would greatly improve the user experience. Content creators could view their schedule on a visual calendar grid and view interactive charts showing post engagement over time.

5.3 Connecting Directly to Social Media APIs

Integrating official social media application programming interfaces (APIs) for Instagram, TikTok, and X would allow the program to post scheduled content automatically at the designated time and automatically fetch updated likes, comments, and views in the background.

5.4 Smart Analytics and Recommendations

Upgrading the analytical features could provide content creators with helpful insights, such as calculating engagement rates, identifying which days of the week get the most views, and suggesting the best times to post content.

---

6. Conclusion

The Social Media Content Planner provides a complete, reliable, and easy-to-use solution for managing social media content. It successfully meets all requirements of the programming principles module by combining clear program structure, data persistence, and thorough input validation. The system serves as a strong starting point that can easily be expanded with modern databases, web interfaces, and automated features in the future.